*Heaven's Light is Our Guide*

Rajshahi University of Engineering & Technology


Department of Electrical & Computer Engineering

Course No.: ECE-2112
Course Title: Digital Techniques Sessional

**Experiment No. :** 01

**Submitted by:**
Nabil Ahmed Arnob
Roll: 2410035

**Submitted to:**
MST. Mazeda Noor Tasnim
Lecturer
Dept. of Electrical & Computer Engineering, RUET

**Date of Experiment :** 19 July 2026
**Date of Submission :** 2 August 2026

## Introduction to Logisim Evolution:

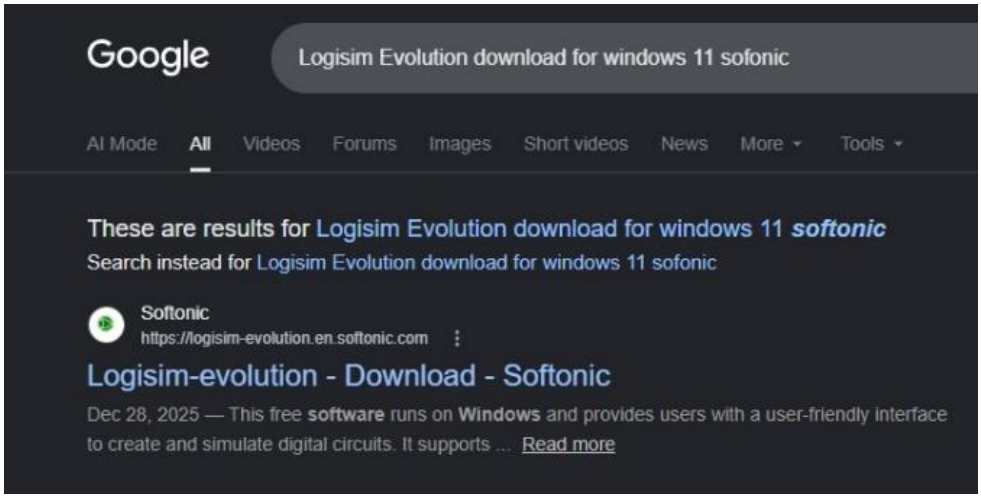
Logisim is a free, open-source digital logic circuit simulation software developed for designing, simulating, and testing digital circuits. It provides a graphical interface where users can build logic circuits using basic and advanced digital components such as logic gates, multiplexers, flip-flops, counters, registers, and memory units. Logisim allows users to verify circuit behavior without requiring physical hardware, making it an effective tool for learning and designing digital systems.

## Uses of Logisim

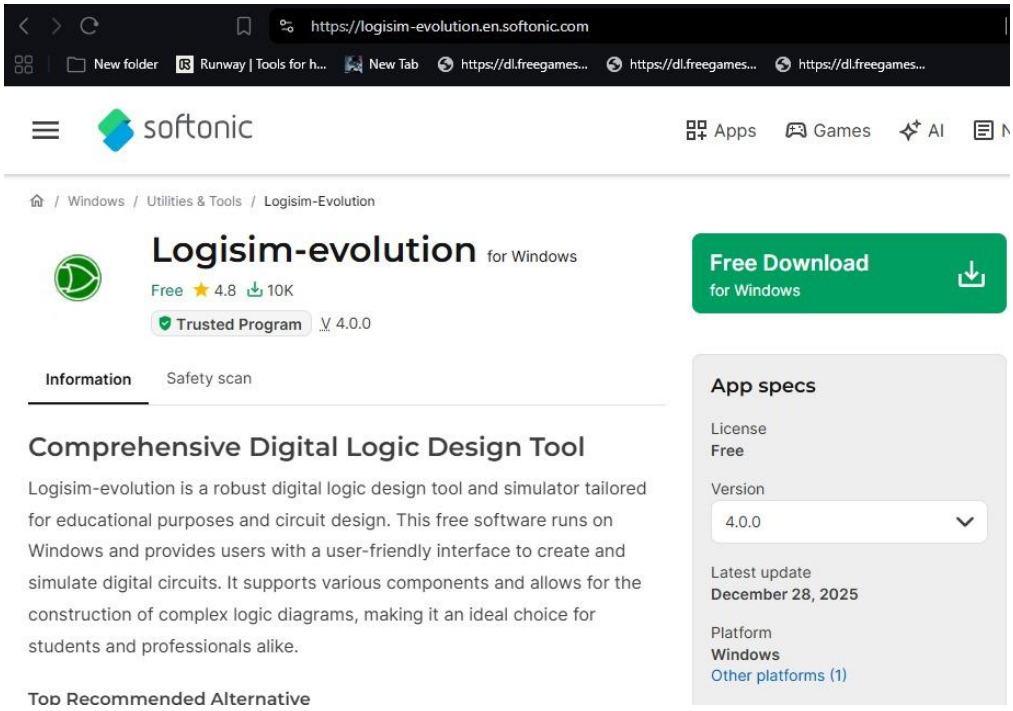
- To design and simulate digital logic circuits.
- To verify the operation of logic gates such as AND, OR, NOT, NAND, NOR, XOR, and XNOR.
- To implement combinational and sequential logic circuits.
- To design arithmetic circuits such as Half Adders, Full Adders, and Subtractors.
- To test circuit outputs using switches, LEDs, and input/output pins.
- To reduce hardware costs by testing circuits before physical implementation.
- To support teaching and learning in Digital Electronics and Computer Architecture courses.

## Installation Process:

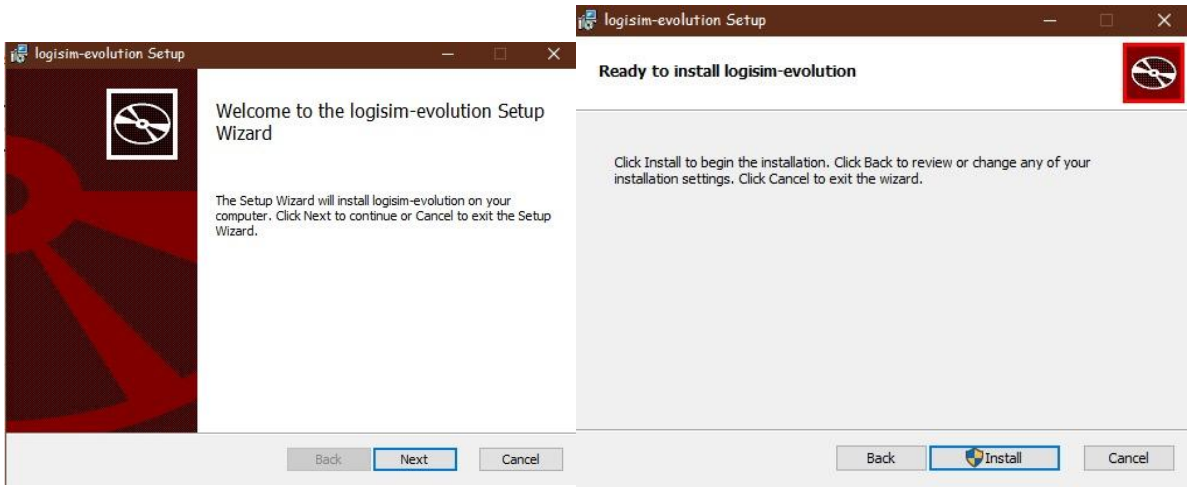
### 1.Search in browser:

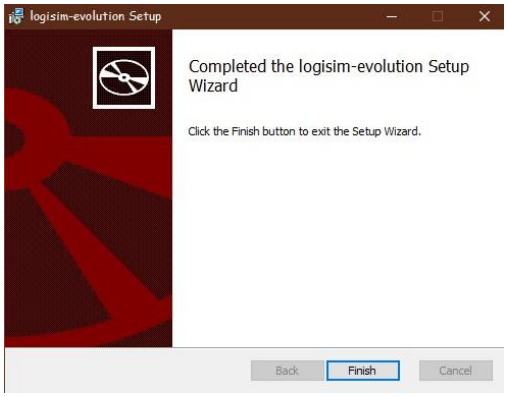


## 2. This is the software I have to download:

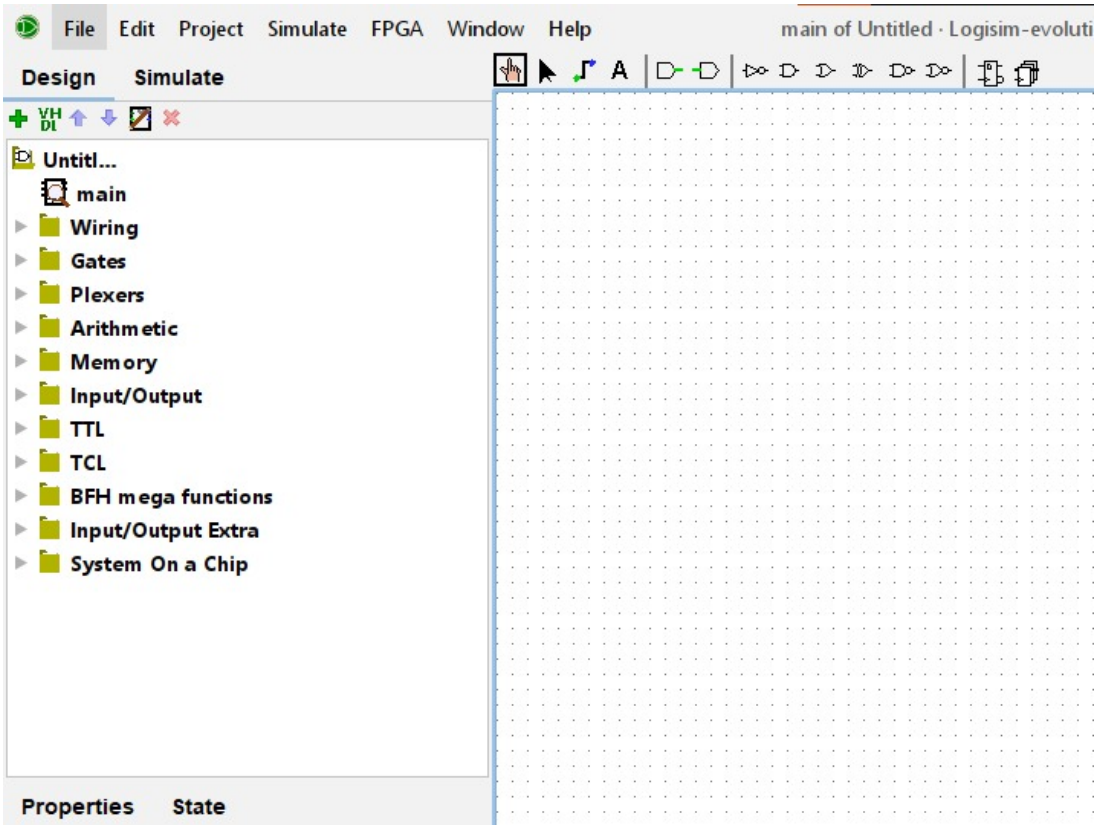



## 3. Then I have to follow the following steps for setup:



## 4. Here this is Logisim Evolution software:



## Problem-01: Implementation of an OR gate using only NOR gates.

### Objective:

To design and verify an OR gate using two NOR gates, and to observe how universal gates can replicate the functionality of other basic gates.

### Theory:

A NOR gate is a universal gate, meaning any logic gate can be implemented using only NOR gates.
The Boolean expression of a NOR gate is:

$$\overline{A + B}$$

To obtain the OR function, the output of the first NOR gate is passed through another NOR gate with both its inputs connected together. This second NOR gate acts as a NOT gate.

$$Y = \overline{\overline{A + B}} = A + B$$

Thus, an OR gate can be implemented using two NOR gates.

### Truth Table:

| Input A | Input B | NOR Output $\overline{A + B}$ | Final Output (OR) A+B |
|---|---|---|---|
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 |

### Circuit Diagram:

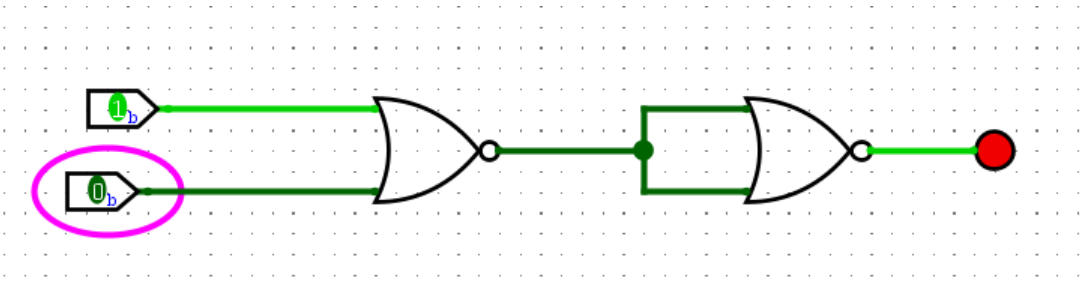


*Figure 1: OR using NOR*

### Discussion:

NOR gate is called a universal gate because it can implement all basic logic gates. Two NOR gates are required to implement one OR gate. The second NOR gate acts as an inverter by connecting both its inputs together. The experimental results matched the theoretical truth table.

### Conclusion:

The experiment successfully demonstrated that an OR gate can be implemented using only NOR gates, proving that the NOR gate is a universal logic gate.

## Problem-02: Implementation of an AND gate using only NOR gates.

### Objective:

To implement an AND gate using only NOR gates and verify its truth table.

### Theory:

A NOR gate is a universal logic gate, meaning it can be used to implement any other logic gate. To realize an AND gate using only NOR gates, the inputs are first inverted using NOR gates configured as NOT gates. These inverted inputs are then applied to another NOR gate.

Using De Morgan's theorem:

$$A.B = \overline{\bar{A} + \bar{B}}$$

Thus, an AND gate can be implemented using three NOR gates.

### Truth Table:

| A | B | $\overline{A}$ | $\overline{B}$ | Final Output (AND) A . B |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 |

### Circuit Diagram:

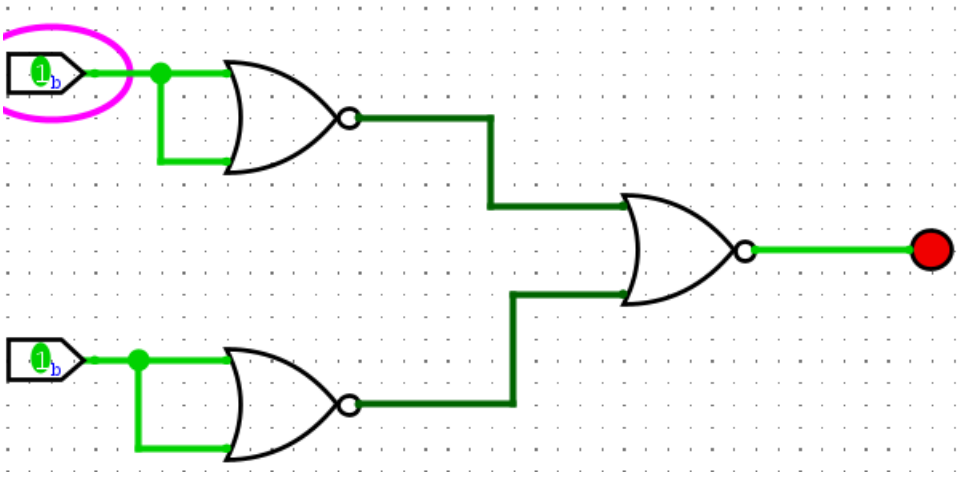


*Figure 2: AND using NOR*

### Discussion:

NOR is a universal gate and can implement all logic gates. Two NOR gates are used as NOT gates to invert the inputs. The third NOR gate combines the inverted inputs to produce the AND function using De Morgan's theorem. The experimental results matched the theoretical outputs.

### Conclusion:

The experiment successfully demonstrated that an AND gate can be implemented using only NOR gates. The output verified the truth table of an AND gate, proving that the NOR gate is a universal gate capable of realizing the AND operation.

## Problem-03: Implementation of an NOT gate using only NOR gates.

### Objective:

To implement a NOT gate using only a NOR gate and verify its truth table.

### Theory:

A NOR gate is a universal logic gate that can be used to implement any logic function. A NOT gate is obtained by connecting both inputs of a NOR gate together.

If the common input is $A$, the output is:

$$Y = \overline{A + A} = \overline{A}$$

Thus, a single NOR gate can function as a NOT gate

### Truth Table:

| Input A | Output $\overline{A}$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

### Circuit Diagram:

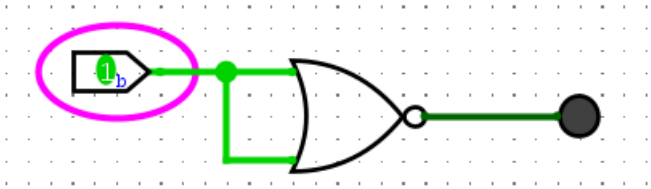


*Figure 3: NOT using NOR*

### Discussion:

A NOR gate acts as a NOT gate when both its inputs are connected together. Only one NOR gate is required to implement a NOT gate. The experimental results verified the Boolean expression $Y = (A + A)' = \bar{A}$. This demonstrates that the NOR gate is a universal gate.

### Conclusion:

The experiment successfully demonstrated the implementation of a NOT gate using only a NOR gate. The obtained output matched the theoretical truth table, confirming the correct operation of the circuit.

## Problem-04: Implementation of an OR gate using only NAND gates.

### Objective:

To implement an OR gate using only NAND gates and verify its truth table.

## Theory:

A NAND gate is a universal logic gate, meaning it can be used to implement any logic function. According to De Morgan's theorem:

$$A+B = \overline{\overline{A} \cdot \overline{B}}$$

The inputs are first inverted using NAND gates configured as NOT gates, and then a third NAND gate combines the inverted inputs to produce the OR output. Thus, an OR gate can be implemented using three NAND gates.

## Truth Table:

| A | B | $\overline{A}$ | $\overline{B}$ | Final Output (OR) A+B |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |

## Circuit Diagram:

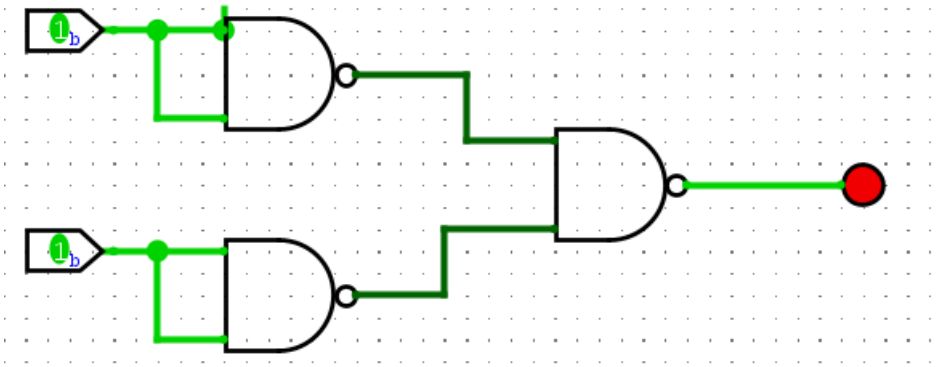


*Figure 4: OR using NAND*

## Discussion:

NAND is a universal gate and can implement all logic gates. Two NAND gates are used as NOT gates by connecting their inputs together. The third NAND gate combines the

inverted inputs to produce the OR function using De Morgan's theorem. The experimental results matched the theoretical truth table.

### Conclusion:

The experiment successfully demonstrated that an OR gate can be implemented using only NAND gates. The obtained output matched the OR gate truth table, confirming that the NAND gate is a universal gate capable of realizing the OR operation.

## Problem-05: Implementation of an AND gate using only NAND gates.

### Objective:

To implement an AND gate using only NAND gates and verify its truth table.]

### Theory:

A NAND gate is a universal logic gate, meaning it can be used to implement any logic function. A NAND gate produces the complement of the AND operation:

$$Y = \overline{A \,.\, B}$$

To obtain an AND gate, the output of the first NAND gate is inverted using a second NAND gate with its two inputs connected together.

Expression:

$$Y = \overline{\overline{A \,.\, B}} = A \,.\, B$$

Thus, an AND gate can be implemented using two NAND gates.

### Truth Table:

| A | B | NAND Output $\overline{A \,.\, B}$ | Final Output (AND) A . B |
|---|---|---|---|
| 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

### Circuit Diagram:

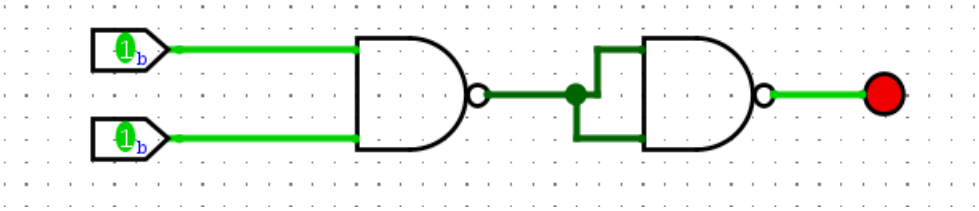


*Figure 5: AND using NAND*

### Discussion:

NAND is a universal gate and can implement all basic logic gates. Two NAND gates are required to implement an AND gate. The second NAND gate acts as an inverter by connecting both its inputs together. The experimental results matched the theoretical truth table.

### Conclusion:

The experiment successfully demonstrated that an AND gate can be implemented using only NAND gates. The obtained output matched the AND gate truth table, confirming that the NAND gate is a universal gate capable of realizing the AND operation.

## Problem-06: Implementation of an NOT gate using only NAND gates.

### Objective:

To implement a NOT gate using only a NAND gate and verify its truth table.

### Theory:

A NAND gate is a universal logic gate that can be used to implement any logic function. A NOT gate is obtained by connecting both inputs of a NAND gate together. If the common input is $A$, the output is:

$$Y= \overline{A \cdot A} = \overline{A}$$

Thus, a single NAND gate can function as a NOT gate.

## Truth Table:

| Input A | Output $\overline{A}$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

## Circuit Diagram:

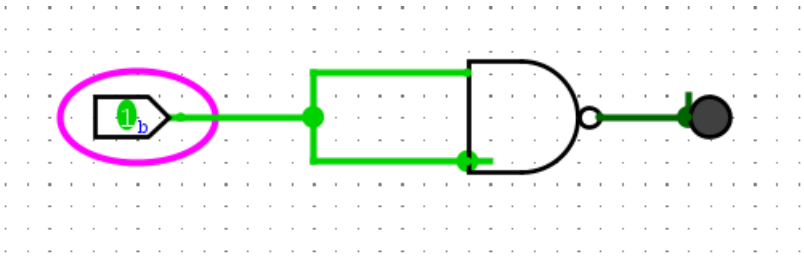


*Figure 6: NOT using NAND*

## Discussion:

In this experiment, a NOT gate was successfully implemented using a single NAND gate by connecting both of its inputs together. The gate produced the complement of the input signal, and the observed outputs matched the expected truth table, confirming the correct operation of the circuit.

## Conclusion:

The experiment successfully demonstrated the implementation of a NOT gate using only a NAND gate. The obtained output matched the theoretical truth table, confirming the correct operation of the circuit

## Problem-07: Implementation of a full adder circuit and testing it against its truth table.

### Objective:

To implement a Full Adder circuit using logic gates and verify its operation by comparing the outputs with its truth table.

### Theory:

A full adder is a combinational logic circuit that adds three 1-bit binary inputs: two operands (A and B) and a carry-in (Cin). It produces two outputs: Sum (S) and Carry-out (Cout).

The Boolean expressions are:

- Sum: $S = A \oplus B \oplus C_{in}$
- Carry: $C_{out} = AB + (A \oplus B)\, C_{in}$

A full adder can be implemented using two half adders and one OR gate. It is widely used in binary addition circuits and arithmetic logic units (ALUs).

### Truth Table:

| A | B | Cin | Sum(S) | Carry(Cout) |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

### Circuit Diagram:

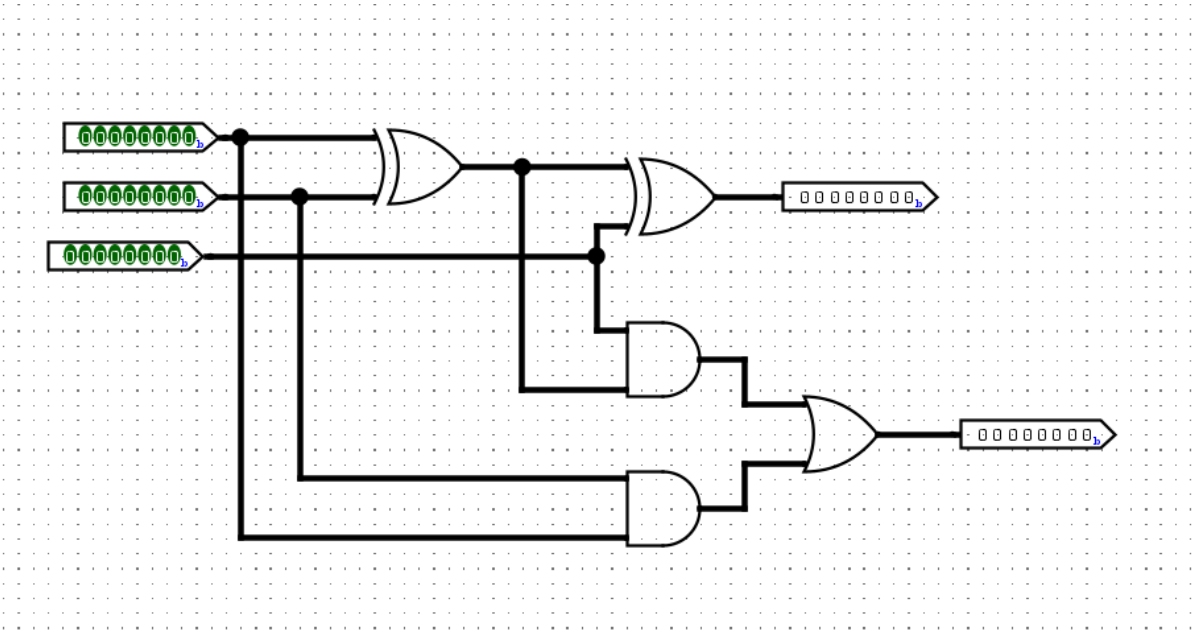


*Figure 7: Full Adder Circuit*

### Discussion:

In this experiment, a full adder circuit was constructed and tested for all possible input combinations of A, B, and Cin. The outputs Sum and Carry-out were observed and compared with the theoretical truth table. The experimental results matched the expected outputs, confirming the correct operation of the full adder circuit.

### Conclusion:

The Full Adder circuit was successfully implemented and tested. The outputs obtained for all input combinations agreed with the theoretical truth table, verifying the correct operation of the Full Adder circuit.

## Problem-08: Implementation of a binary to BCD converter fora 4-bit binary input.

### Objective:

To implement a 4-bit Binary to BCD (Binary-Coded Decimal) converter and verify its operation by comparing the outputs with the expected BCD values.

## Theory:

A Binary to BCD (Binary-Coded Decimal) converter converts a binary number into its equivalent BCD representation. In this experiment, an 8-bit binary number (0–255) is applied to the converter. The output is displayed on three BCD displays representing the hundreds (100), tens (10), and ones (1) digits. Each decimal digit is encoded separately as a 4-bit BCD code, making the output suitable for digital displays and numerical processing.

## Truth Table:

| Binary input | Decimal output | Hundreds | Tens | Ones |
|---|---|---|---|---|
| 00000000 | 0 | 0 | 0 | 0 |
| 00000001 | 1 | 0 | 0 | 1 |
| 00001010 | 10 | 0 | 1 | 0 |
| 00011111 | 31 | 0 | 3 | 1 |
| 01100100 | 100 | 1 | 0 | 0 |
| 11101011 | 235 | 2 | 3 | 5 |
| 11111111 | 255 | 2 | 5 | 5 |

## Circuit Diagram:

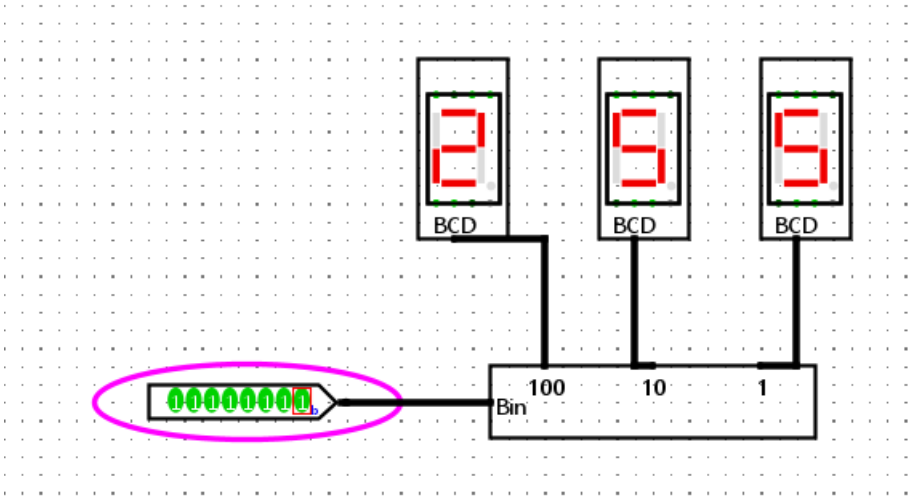



*Figure 8: Binary to BCD*

## Discussion:

The converter transforms a 4-bit binary number into two BCD digits. Numbers **0–9** require only the ones BCD digit, while **10–15** require both tens and ones BCD digits. The circuit is useful for interfacing binary systems with decimal display devices such as seven-segment displays.

## Conclusion:

The Binary to BCD converter was successfully implemented and tested for all 4-bit binary inputs. The output BCD values agreed with the expected truth table, confirming the correct operation of the converter circuit.